

Distributed Job Scheduler

Airflow · Temporal · Cron at scale – "jobs ని కాల bucket ల ప్రకారం shard చేసి, ప్రతి shard ని ఒక worker కి ఇద్దాం" అని అందరూ మొదలుపెడతారు.

మూడు విరుపులు. మొదటిది – మీ jobs lo **23.5% సరిగ్గా అర్థరాత్రి** నడుస్తాయి, కాబట్టి కాలం ప్రకారం shard చేస్తే అసమతుల్యత **2 కోట్ల రెట్లు**. రెండోది – shard యజమాని మారినప్పుడు **1.02% jobs రెండుసార్లు** నడుస్తాయి. మూడోది – "రోజూ ఉదయం 9" అనే job **365 lo 220 రోజులు** తప్పు సమయంలో నడుస్తుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` lo run చేసినదే.

LLD DEEP 25 (JOB SCHEDULER) LO RETRY, LEASE, CATCH-UP ఉన్నాయి – వాటిని ఇక్కడ వాడతాను, మళ్ళీ నిరూపించను

అక్కడ కొలిచినవి: jitter లేని ఘాతీయ backoff ఒక్క పీలుపు కూడా ఆదా చేయలేదు; heartbeat లేని lease **16.4% jobs** ని రెండుసార్లు నడిపింది; fencing లేకపోతే **2.65% jobs** పాత worker ఫలితాన్ని పట్టుకున్నాయి; మరియు సరైన ఒక్క catch-up విధానం లేదు.

ఆ నాలుగూ ఇక్కడా వర్తిస్తాయి, మరియు ఈ doc వాటిని ఒక పునాదిగా తీసుకుంటుంది.

ఈ doc అడిగేది ఒక్క ప్రశ్న, మరియు అది LLD 25 అడగనిది:

"10 కోట్ల jobs వందల machines మీద ఉంటే – ఏ job ఏ machine కి చెందుతుంది?"

సహజమైన జవాబు "కాలం ప్రకారం విభజించు" – మరియు §5 lo కొలుస్తాం, అది సరిగ్గా అత్యంత చెడ్డ ఎంపిక, ఎందుకంటే కాలమే skew ఉన్న చోటు.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: అర్థరాత్రి

3. Step – కాల buckets, ఒక్కో shard కి కొన్ని
4. మొదటి విరుపు – 23.5% jobs ఒకే సెకనులో
5. Step – కాలం కాదు, jobId ప్రకారం

Part 3 – రెండో విరుపు: యజమాని మారినప్పుడు

6. Step – ఒక్కో shard కి ఒక యజమాని
7. రెండో విరుపు – 1.02% jobs రెండుసార్లు
8. Step – యుగం (epoch) · lease సరిపోదు

Part 4 – మూడో విరుపు: "రోజూ ఉదయం 9"

9. Step – తర్వాతి పరుగు = చివరిది + 24 గంటలు
10. మూడో విరుపు – 365 lo 220 రోజులు తప్పు
11. Step – ప్రతిసారీ క్యాలెండర్ నుంచి

Part 5 – పూర్తి system

12. మొత్తం code · 4.2 లక్షల job runs · mutation testing

Part 6 – Interview lo

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

14. నోటితో చెప్పాల్సిన English script

15. Follow-ups – dependencies, priorities, బహుళ ప్రాంతాలు

16. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు

10 కోట్ల scheduled jobs. ప్రతి సెకనుకీ "ఇప్పుడు ఏవి నడపాలి?" అని తెలుసుకుని, వాటిని నడపాలి. వందల machines మీద, వాటిలో ఏదైనా ఎప్పుడైనా చనిపోవచ్చు.

మొదట ఒక సహజమైన భయాన్ని కొలిచి చూద్దాం: ఆ "ఇప్పుడు ఏవి due?" అనే query ఖరీదైనదేనా?

100 మిలియన్ scheduled jobs · ప్రతి సెకనుకీ ఒక tick
30 రోజుల్లో సమానంగా పంచితే → సెకనుకీ సగటున 39 jobs due

సెకనుకీ 39. ఒక B-tree index అయినా, కాల bucket అయినా – రెండూ అదే 39 rows చదువుతాయి. ఇది ఒక సమస్య కాదు.

కానీ ఆ లెక్కలో ఒక పదం ఉంది, మరియు అదే మొత్తం **PROBLEM**

"సమానంగా పంచితే."

Jobs సమానంగా పంచబడవు. §4 lo చూపిస్తాను – **86,400 సెకన్లలో 84,137 (97.4%) పూర్తిగా ఖాళీ,** మరియు **23.5% jobs సరిగ్గా ఒకే సెకనులో** ఉన్నాయి.

అంటే "సెకనుకీ 39" అనేది ఎప్పుడూ జరగని సగటు. నిజం – 97.4% సెకన్లలో **సున్నా**, మరియు ఒక సెకనులో **2.35 కోట్లు**.

కాబట్టి ఈ problem యొక్క నిజమైన ప్రశ్నలు మూడు, మరియు మూడూ **పంపిణీ** గురించి – పరిమాణం గురించి కాదు.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
Jobs ఏ సమయాల్లో షెడ్యూల్ అవుతాయి?	ఇదే §4. "గుండ్రని సమయాలు" అంటే మీ shard వ్యాసం మారుతుంది
ఒక job రెండుసార్లు నడిస్తే ఫరవాలేదా?	లేదంటే §8 యొక్క epoch తప్పనిసరి
ఒక job ఆలస్యంగా నడిస్తే ఫరవాలేదా?	అర్థరాత్రి 2.35 కోట్లు – ఏదో ఒకటి ఆగాల్సిందే
స్థానిక కాలమానం ముఖ్యమా?	అవును అంటే DST – §10 lo 220/365 రోజులు తప్పు
Jobs ఒకదాని మీద ఒకటి ఆధారపడతాయా?	అవును అంటే ఇది scheduler కాదు, workflow engine – §15
Worker చనిపోతే ఎంత త్వరగా తెలియాలి?	lease వ్యవధి – §7
జాబితా ఎంత తరచుగా మారుతుంది?	అరుదుగా అయితే shard map ని cache చేయొచ్చు

ఒక ప్రశ్న అడిగితే మరు ఈ SYSTEM ని నిజంగా నడిపారని తెలుస్తుంది

"మీ jobs lo ఎన్ని శాతం అర్థరాత్రి లేదా గంట మొదట్లో నడుస్తాయి?"

ఎందుకంటే అది ఒక capacity ప్రశ్న కాదు – అది ఒక **sharding** ప్రశ్న. §4 lo కొలుస్తాం: ఆ జవాబు "చాలా ఎక్కువ" అయితే, కాలం ప్రకారం shard చేయడం 2 కోట్ల రెట్ల అసమతుల్యత తెస్తుంది.

మరియు ఆ అసమతుల్యత shards పెంచితే ఫిరమవుతుంది, తగ్గదు.

Part 2 – మొదటి విరుపు: అర్థరాత్రి

3. Step – కాల buckets, ఒక్కో shard కి కొన్ని

"ఇప్పుడు ఏవి due?" అని వేగంగా తెలుసుకోవాలంటే సహజమైన నిర్మాణం **కాల bucket**: ప్రతి నిమిషానికి ఒక partition, అందులో ఆ నిమిషంలో నడవాల్సిన jobs.

```
bucket(12:00) → [job1, job7, job99, ...]
bucket(12:01) → [job3, ...]
```

ఇప్పుడు shard చేయాలి. **అత్యంత సహజమైన ఎంపిక: buckets ని shards మధ్య పంచడం.**

```
shard = bucket % NUM_SHARDS;
```

ఇది చక్కగా అనిపిస్తుంది: ఒక్కో shard ఒక్కో కాల ముక్కకి బాధ్యత వహిస్తుంది, మరియు అవి ఒకదానితో ఒకటి పోటీ పడవు.

4. మొదటి విరుపు – 23.5% jobs ఒకే సెకనులో

నిజమైన cron fleets లో ఎవరూ "17వ నిమిషం" అని రాయరు. అందరూ **గుండ్లని సమయాలు** రాస్తారు – అర్థరాత్రి, గంట మొదలు, :15, :30.

100 మిలియన్ jobs • ఒక రోజులో ఎక్కడ పడతాయి?

సమయం	ఆ ఒక్క సెకనులో jobs	మొత్తంలో %
00:00:00	2,35,27,778	23.5%
01:00:00	40,99,206	4.1%
02:00:00	40,99,206	4.1%
03:00:00	40,99,206	4.1%
06:00:00	40,99,206	4.1%

jobs ఉన్న సెకన్లు : 2,263 / 86,400 (2.6%)
ఖాళీ సెకన్లు : 84,137 (97.4%)

86,400 సెకన్లలో 84,137 పూర్తిగా ఖాళీ • ఒక సెకనులో 2.35 కోట్లు

§1 లో లెక్కించిన "సెకనుకి 39" అనే సగటు – అది **ఎప్పుడూ జరగదు.**

జరిగేది: 97.4% సమయం ఏమీ లేదు, మరియు కొన్ని క్షణాల్లో అంతా ఒకేసారి.

ఇప్పుడు ఆ పంపిణీని $\text{shard} = \text{bucket} \% N$ తో కలిపితే?

అతి పెద్ద సెకను: 2,35,27,778 jobs ఒకేసారి

shards	shard కి సగటున	అతి పెద్ద shard కి	అసమతుల్యత
16	72	2,35,27,778	$3,25,248\times$
64	18	2,35,27,778	$13,00,992\times$
256	5	2,35,27,778	$52,03,968\times$
1024	1	2,35,27,778	$2,08,15,872\times$

SHARDS పెంచితే అసమతుల్యత పెరుగుతుంది

16 shards → 3.25 లక్షల రెట్లు. 1,024 shards → **2.08 కోట్ల రెట్లు**.

ఎందుకంటే అర్థరాత్రి అనేది **ఒకే bucket**, కాబట్టి అది **ఒకే shard** మీద పడుతుంది – మీరు ఎన్ని shards వేసినా. మిగతా 1,023 ఖాళీగా కూర్చుని చూస్తాయి.

Shards పెంచడం సాధారణంగా భారాన్ని పంచుతుంది. ఇక్కడ అది కేవలం ఖాళీ machines ని పెంచుతుంది.

5. Step – కాలం కాదు, jobId ప్రకారం

తప్పు ఎక్కడ జరిగిందో ఇప్పుడు స్పష్టం:

SHARD KEY ఎన్నుకునే నియమం

Shard key ఏకరీతిగా పంచబడిన దాని మీద ఉండాలి. కాలం ఏకరీతిగా పంచబడదు – అదే skew ఉన్న చోటు.

Job ids ఏకరీతిగా పంచబడతాయి (hash చేస్తే). కాబట్టి $\text{hash}(\text{jobId}) \% N$ ప్రకారం shard చేసి, **ప్రతి shard తన సొంత కాల buckets** ని ఉంచుకోవాలి.

అప్పుడు అర్థరాత్రి 2.35 కోట్లు **అన్ని shards మీదా సమానంగా** పడతాయి.

```
// hash - jobId నుంచి shard. కాలం *కాదు*: కాలమే skew ఉన్న చోటు (§5).
function shardOf(jobId, shards) {
  let h = 2166136261;
  for (let i = 0; i < jobId.length; i++) {
    h ^= jobId.charCodeAt(i); h = Math.imul(h, 16777619);
  }
  return ((h >>> 0) % shards);
}
```

పరిష్కారం: jobId ప్రకారం shard · ప్రతి shard తన సొంత buckets

shards | అర్థరాత్రి ఒక్కో shard కి | అసమతుల్యత | ఒక్కో shard ఎంతసేపు పని

16	14,70,486	1.0x	294 సె
64	3,67,622	1.0x	74 సె
256	91,905	1.0x	18 సె
1024	22,976	1.0x	5 సె

2,08,15,872x → 1.0x, మరియు ఇప్పుడు shards పెంచడం **నిజంగా సహాయపడుతుంది** – 294 సెక్స్ నుంచి 5 సెక్స్ కి.

ఇది HLD DEEP 06 §5 యొక్క అదే ఆలోచన, వేరే వేషంలో

అక్కడ: ఒక 5.6-రీజల transcode job ని **ముక్కలు చేయకపోతే** 2,048 machines పనికీరావు.

ఇక్కడ: అర్ధరాత్రి భారాన్ని **సరైన కీ మీద పంచకపోతే** 1,024 shards పనికీరావు.

రెండింటిలోనూ: మీ సమాంతరత మీ machines సంఖ్య మీద కాదు, మీ విభజన మీద ఆధారపడుతుంది.

Part 3 – రెండో విరుపు: యజమాని మారినప్పుడు

6. Step – ఒక్కో shard కి ఒక యజమాని

Shards ఉన్నాయి. ఇప్పుడు వాటిని workers కి ఇవ్వాలి – మరియు ఒక్కో shard కి ఒక్కడే యజమాని, లేకపోతే ఇద్దరూ అదే jobs నడుపుతారు.

యజమాని చనిపోతే ఇంకొకరు తీసుకోవాలి. అది ఎలా తెలుస్తుంది? **Heartbeat ఆగితే.**

LLD Deep 25 §8 lo ఈ నమూనాని కొలిచాం: heartbeat + lease. అది job స్థాయిలో పని చేసింది.

ఇక్కడ అది **shard** స్థాయిలో. ఒక shard lo వేల jobs ఉన్నాయి, కాబట్టి ఒక తప్పు handover వేల jobs ని ప్రభావితం చేస్తుంది.

7. రెండో విరుపు – 1.02% jobs రెండుసార్లు

256 shards · 32 workers · 2,00,000 shard-ticks

lease 10 సె · heartbeat 3 సె · 1.5% ticks lo యజమాని ఆగిపోతాడు

విధానం	handovers	రెండుసార్లు నడిచినవి	దాటేసినవి
lease లేకుండా	3,046	2,049 (1.02%)	989
lease ఉంది · fencing లేదు	3,046	146 (0.07%)	0
lease + fencing (epoch)	3,034	0 (0.00%)	0

LEASE ఒక్కటే 1.02% నుంచి 0.07% కి తెచ్చింది – కానీ సున్నాకి కాదు

ఆ మిగిలిన 0.07% ఒక **ఇరుకైన కిటికీ**: పాత యజమాని తన lease చూసుకున్నాడు (సజీవం), తర్వాత job నడపబోయే క్షణంలో lease గడువు తీరింది.

"చూసిన తర్వాత, చేసే ముందు" – ఆ అంతరం ఎప్పుడూ ఉంటుంది. దాన్ని **చిన్నది** చేయొచ్చు, **తీసేయలేం.**

146 shard-handovers × ఒక్కో shard lo వేల jobs = **లక్షల jobs రెండుసార్లు.**

8. Step – యుగం (epoch) • lease సరిపోదు

పరిష్కారం LLD Deep 25 §8 lo నేర్చుకున్నదే, కానీ ఇక్కడ **shard స్థాయిలో**: ప్రతి ownership కి ఒక **యుగం (epoch)**, మరియు ప్రతి చర్య ఆ యుగాన్ని చూపించాలి.

```
// §8 – shard కి ఒక యజమాని • ఒక యుగం (epoch). పాత యుగం రాయలేదు.  
claim(worker, shard) {  
  const now = this.#clock(), cur = this.#owner.get(shard);  
  if (cur && cur.worker !== worker &&  
    now - cur.lastBeat <= this.heartbeatMs + this.leaseMs) {  
    this.stats.denied++;  
    return { ok: false, reason: 'HELD' };  
  }  
  if (cur && cur.worker !== worker) this.stats.handovers++;  
  const epoch = ++this.#epoch;  
  this.#owner.set(shard, { worker, epoch, lastBeat: now });  
  this.stats.claimed++;  
  return { ok: true, epoch };  
}
```

మరియు **రెండు** చోట్ల ఆ యుగం పరీక్షించాలి – jobs అడిగేటప్పుడు, మరియు పూర్తయిందని చెప్పేటప్పుడు:

```
// §8 – పూర్తయిందని చెప్పేటప్పుడు యుగం సరిపోవాలి. లేకపోతే *తిరస్కరణ*.  
complete(shard, epoch, jobId) {  
  const cur = this.#owner.get(shard);  
  if (!cur || cur.epoch !== epoch) {  
    this.stats.staleReject++;  
    return { ok: false, reason: 'STALE_EPOCH' };  
  }  
  ...  
}
```

ఎందుకు రెండు చోట్లా – ఒక్కటి సరిపోదా?

సరిపోదు, మరియు కారణం సూక్ష్మమైనది.

ఒక worker `due()` పిలిచి job ids పొందాడు. ఆ తర్వాత అతను ఆగిపోయాడు. ఈలోపు handover జరిగింది. అతను మేల్కొని ఆ **పాత ids** తో `complete()` పిలుస్తాడు.

`due()` lo మాత్రమే పరీక్షిస్తే – అది అప్పుడు సరైనదే, కాబట్టి ఆగదు. `complete()` lo పరీక్ష లేకపోతే అతని ఫలితం అంగీకరించబడుతుంది.

§12 lo ఈ మార్పు **262/300** సార్లు పట్టుబడింది – కానీ **నా పరీక్ష దాన్ని ముందు పట్టుకోలేదు**, ఎందుకంటే నా fuzz job ids ని handover అంతటా పట్టుకుని ఉంచలేదు. దాన్ని సరిచేశాకే అది కనిపించింది.

Part 4 – మూడో విరుపు: "రోజూ ఉదయం 9"

9. Step – తర్వాతి పరుగు = చివరిది + 24 గంటలు

Job నడిచాక తర్వాతి పరుగు ఎప్పుడో లెక్కించాలి. "రోజూ ఒకసారి" అంటే సులభం:

```
nextRun = lastRun + 24 * 60 * 60 * 1000;
```

ఇది సరైనది – DST లేని ప్రాంతాల్లో.

DST ఉన్న ప్రాంతాల్లో ఒక రోజు 23 గంటలు, ఇంకొక రోజు 25 గంటలు. అప్పుడు?

10. మూడో విరుపు – 365 to 220 రోజులు తప్పు

"ప్రతిరోజూ స్థానిక కాలమానంలో ఉదయం 9" • ఒక సంవత్సరం • రెండు DST మార్పులు

విధానం	తప్పు సమయంలో నడిచిన రోజులు	DST చుట్టూ స్థానిక గంటలు
చివరి పరుగు + 24 గంటలు	220 / 365	9, 10, 10
ప్రతినాటి క్యాలెండర్ నుంచి	0 / 365	9, 9, 9

220 రోజులు – ఒక్క రోజు కాదు

చాలామంది అనుకుంటారు: "DST రోజున ఒక్కసారి తప్పు అవుతుంది, ఫరవాలేదు."

కాదు. DST మార్పు తర్వాత job ఒక గంట జరిగిపోతుంది, మరియు అలాగే ఉండిపోతుంది – తర్వాతి DST మార్పు వరకు. అంటే సంవత్సరంలో సగం కంటే ఎక్కువ.

"ఉదయం 9 కి report పంపు" అనేది ఉదయం 10 కి పంపబడుతుంది, ఆరు నెలలు.

పరిష్కారం: తర్వాతి పరుగుని ఎప్పుడూ క్యాలెండర్ నుంచి లెక్కించడం, గత పరుగు నుంచి కాదు.

```
// §11 – తర్వాతి పరుగు ఎప్పుడూ *క్యాలెండర్* నుంచి, "చివరిది + విరామం" నుంచి కాదు
#next(spec, after) {
  if (spec.kind === 'interval') {
    // విరామాలకి క్యాలెండర్ అవసరం లేదు
    const n = Math.floor(after / spec.everyMs) + 1;
    return n * spec.everyMs;
  }
}
```

```
// రోజువారీ: ఆ ప్రాంతం యొక్క ఆ రోజు offset తో లెక్కించడం
let d = Math.floor(after / DAY);
for (let i = 0; i < 3; i++) {
  const t = d*DAY + (spec.hour - spec.offsetFor(d))*HOUR;
  if (t > after) return t;
  d++;
}
return d*DAY + (spec.hour - spec.offsetFor(d))*HOUR;
}
```

11. Step – ప్రతిసారీ క్యాలెండర్ నుంచి

కానీ DST కి రెండు అంచు పరిస్థితులు ఉన్నాయి, మరియు వాటికి "సరైన" జవాబు లేదు:

వసంతంలో : 2:00 → 3:00 కి దూకుతుంది • "2:30 కి నడపు" అనే job కి
ఆ రోజు 2:30 *ఉనికిలోనే లేదు*
శరత్తులలో : 2:00 → 1:00 కి తిరిగి వస్తుంది • "1:30 కి నడపు" అనే job
ఆ రోజు 1:30 *రెండుసార్లు* వస్తుంది

ఈ రెండు ENGINEERING ప్రశ్నలు కావు – అవి PRODUCT నిర్ణయాలు

లేని సమయం → దాటేయాలా, లేక 3:00 కి నడపాలా?

రెండుసార్లు వచ్చే సమయం → రెండుసార్లు నడపాలా, లేక మొదటిసారి నడపాలా?

"రోజువారీ report" కి – దాటేయడం తప్పు (ఒక రోజు report లేదు), రెండుసార్లు నడపడం తప్పు (రెండు reports).

HLD Deep 01 §11 మరియు HLD Deep 05 §11 లో చూసిన అదే ఆకారం: వ్యవస్థకి తెలియని నిర్ణయాన్ని వ్యవస్థ తనుకోకూడదు – దాన్ని job నిర్వచనంలో మోసుకెళ్ళాలి.

Part 5 – పూర్తి system

12. మొత్తం code · 4.2 లక్షల job runs · mutation testing

ఏడు నియమాలు

#	నియమం	విరిగితే అర్థం
1	<code>epoch</code> ఎప్పుడూ పెరగాలి	ownership గందరగోళం
2	ఒకే (job, nextRun) రెండుసార్లు పూర్తి కాకూడదు	§8 రక్షణ లేదు
3	<code>nextRun</code> ఎప్పుడూ ముందుకే	§11 విరిగింది
4	ప్రతి job కి చెల్లుబాటుయ్యే <code>nextRun</code>	job మాయమైంది
5	Shard భారం ఏకరీతిగా	§5 విరిగింది
6	యజమాని ఉన్న shard lo overdue job తప్పక కనిపించాలి	job శాశ్వతంగా పోయింది
7	సజీవ lease ని ఎవరూ లాక్చేకూడదు	§8 lease విరిగింది

నియమం 6 ఒక శాశ్వత నష్టాన్ని పట్టుకుంది

నా మొదటి `due()` ఇలా ఉండేది – ప్రస్తుత bucket మరియు దాని ముందు రెండు:

```
for (let b = nowB - 2; b <= nowB && out.length < limit; b++) { ... }
```

అది సహేతుకంగా అనిపిస్తుంది: కొంచెం ఆలస్యాన్ని భరించడానికి ఒక చిన్న కిటికీ.

కానీ ఒక shard కి **యజమాని లేని సమయం** ఆ కిటికీ కంటే ఎక్కువ అయితే?

```
20 సెకన్ల తర్వాత (17 buckets) · due: []  
job యొక్క nextRun: 1000 (ఇది 20000 కంటే చాలా ముందుది)
```

ఆ JOB ఎప్పటికీ కనిపించదు

అది due అయింది. దాని `nextRun` గతంలో ఉంది. కానీ దాని bucket కిటికీ వెనక పడిపోయింది, మరియు కిటికీ ఎప్పుడూ వెనక్కి చూడదు.

Handover ఆలస్యం, deploy, ఒక worker crash – ఇవన్నీ ఆ కిటికీని దాటిస్తాయి. మరియు ప్రతిసారీ **కొన్ని jobs శాశ్వతంగా పోతాయి.**

మౌనంగా. Error లేదు, alert లేదు – ఆ job మళ్ళీ ఎప్పుడూ నడవదు.

పరిష్కారం ఒక **scan floor**: ఒక్కో shard కి "ఇంకా పూర్తిగా ఖాళీ కాని అతి పాత bucket", మరియు అక్కడి నుంచి scan చేయడం.

```
// ఒక్కో shard కి "ఇంకా పూర్తిగా ఖాళీ కాని అతి పాత bucket".
// ఇది లేకపోతే - shard కి యజమాని లేని సమయంలో jobs కిటికీ దాటిపోయి
// *శాశ్వతంగా* మాయమవుతాయి. §13 చూడండి.
#floor = new Map();
```

ఈ ఒక్క మార్పు JOB RUNS ని 2,407 నుంచి 4,18,913 కి తీసుకెళ్ళింది

అదే fuzz, అదే seeds, అదే అడుగులు. తేడా - ఇంతకుముందు jobs కిటికీ దాటిపోయి మాయమవుతూ ఉండేవి.

అంటే నా system 99.4% jobs ని **మౌనంగా పోగొడుతూ ఉంది**, మరియు నా మొదటి ఆరు నియమాలూ దాన్ని **సరైనదిగా** ప్రకటించాయి.

"ఏదీ తప్పు జరగలేదు" అనే నియమాలు సరిపోవు - "ఏదీ జరగాలి" అనే నియమం కూడా కావాలి.

1,000 యాదృచ్ఛిక ప్రయోగాలు · 4,18,913 job runs · 1,74,243 shard claims
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

defined	40,000
due	4,18,913
ran	4,18,913
staleReject	89,018
handovers	1,19,640
claimed	1,74,243
denied	63,217

Test విఫలం కాగలదా? - ఆరు మార్పులు

మార్పు లేని code	→	0/1000 విఫలం
due() lo epoch పరీక్ష తీసేస్తే	→	300/300 విఫలం
ఉదా: shard 2: పాత epoch 3 కి due జాబితా వచ్చింది		
complete() lo epoch పరీక్ష తీసేస్తే	→	262/300 విఫలం
ఉదా: పాత epoch 113 తో j16 పూర్తి చేయగలిగాం		
claim() lo ప్రస్తుత lease గౌరవించకపోతే	→	300/300 విఫలం
ఉదా: shard 8: w1 సజీవంగా ఉండగా w4 లాక్కున్నాడు		
shard ని jobId కి బదులు కాలం ప్రకారం చేస్తే	→	56/300 విఫలం
ఉదా: shard భారం అసమతుల్యం: 0-40		
nextRun ని "చివరిది + విరామం" గా లెక్కిస్తే	→	0/300 విఫలం
scan floor తీసేసి స్థిర కిటికీ వాడితే	→	300/300 విఫలం
ఉదా: shard 7: overdue j19 due() lo రాలేదు		

ఐదు పట్టుబడ్డాయి. **ఐదో మార్పు (after + everyMs) పట్టుబడలేదు - మరియు అది సరైనదే.**

ఎందుకు ఆ మార్పు ఒక BUG కాదు (విరామాలకి)

after + everyMs మరియు grid-aligned రెండూ ముందుకే కదులుతాయి, కాబట్టి ఏ నియమమూ విరగదు.

తేడా **drift**: job ఆలస్యంగా నడిస్తే, "చివరిది + విరామం" ఆ ఆలస్యాన్ని శాశ్వతంగా మోసుకెళ్తుంది. Grid-aligned తిరిగి సరైన క్షణాలకి వస్తుంది.

అది ఒక ఖచ్చితత్వ లక్షణం, ఒక భద్రతా లక్షణం కాదు – కాబట్టి దాని రుజువు §10 పట్టికలో (220/365), fuzz lo కాదు.

మరియు **రోజువారీ** jobs కి అదే తప్పు ఒక భద్రతా సమస్య అవుతుంది, ఎందుకంటే అక్కడ DST ఉంది. అందుకే ఆ రెండు దారులూ code lo వేరుగా ఉన్నాయి.

Part 6 – Interview lo

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెయ్యాలి
0-6	"సెకనుకి 39 jobs – ఇది సమస్య కాదు" అని చూపించి, వెంటనే "కానీ అది సగటు"
6-10	§2 ప్రశ్నలు. ముఖ్యంగా "ఎన్ని శాతం అర్థరాత్రి?"
10-20	మొదటి విరుపు. 23.5% ఒక సెకనులో, 97.4% ఖాళీ. కాలం ప్రకారం shard → 2 కోట్ల రెట్లు. jobId ప్రకారం → 1.0x
20-30	రెండో విరుపు. Handover → 1.02%. Lease → 0.07%. Epoch → 0. రెండు చోట్లా పరీక్షించాలి
30-37	మూడో విరుపు. DST → 220/365. క్యాలెండర్ నుంచి లెక్కించడం. రెండు అంచు పరిస్థితులు
37-45	ఏడు నియమాలు, ముఖ్యంగా నియమం 6 మరియు అది పట్టుకున్న శాశ్వత నష్టం

ఏమి తప్పక చెప్పాలి

1. **Shard key ఏకరీతిగా పంచబడిన దాని మీద ఉండాలి.** కాలం ఏకరీతి కాదు – అదే skew ఉన్న చోటు.
2. కాలం ప్రకారం shard చేస్తే shards పెంచడం అసమతుల్యతని పెంచుతుంది, తగ్గించదు.
3. **Epoch ని రెండు చోట్లా పరీక్షించాలి** – పని తీసుకునేటప్పుడు, మరియు పూర్తయిందని చెప్పేటప్పుడు.
4. "చివరిది + 24 గంటలు" ఒక రోజు తప్పు కాదు – 220 రోజులు తప్పు.
5. **DST యొక్క రెండు అంచు పరిస్థితులకి సరైన జవాబు లేదు** – అది job నిర్వచనంలో ఉండాలి.

ఏమి వదిలేయాలి

- Retry/backoff/jitter – LLD Deep 25 lo కొలిచాం, ఒక వాక్యం
- Lease/heartbeat యొక్క ప్రాథమిక వివరణ – అదే
- Cron వ్యక్తీకరణ parsing
- Leader election యొక్క algorithm (Raft/ZK) – "ఒక coordination service" అని ఒక వాక్యం

14. నోటీతో చెప్పాల్సిన English script

"Let me start by ruling out the thing people usually attack. A hundred million scheduled jobs spread over thirty days is **39 jobs due per second** – a B-tree index or a time bucket both read the same 39 rows. Not a problem. But that number contains the word '**spread**', and that's the whole problem, because they aren't."

"Real cron fleets don't have anyone writing 'seventeen past'. Everyone writes round times. I modelled a realistic mix and measured: **84,137 of 86,400 seconds are completely empty – 97.4% – and 23.5% of all jobs fire in a single second**, midnight. So 'thirty-nine per second' is an average that never happens."

"Now the sharding decision. The natural choice is to shard by time bucket – each shard owns a slice of the clock. That is exactly backwards. Midnight is **one bucket**, so it lands on **one shard**, no matter how many you have. I measured the imbalance: at 16 shards it's 325,000x; at 1,024 shards it's **20.8 millionx**. **Adding shards makes it worse**, because all you're adding is idle machines."

"The rule is: **shard on something uniformly distributed, and time is precisely where the skew lives**. So shard on `hash(jobId)`, and let each shard keep its own time buckets. Imbalance goes to **1.0x**, and now adding shards actually helps – the midnight burst drains in 5 seconds at 1,024 shards instead of 294 at 16."

"Second, ownership. One owner per shard, detected dead by heartbeat – that's the pattern I measured in the LLD version at job granularity. Here it's at shard granularity, so one bad handover affects thousands of jobs at once. Without a lease, **1.02% of shard ticks run twice**. A lease takes it to 0.07%, but not to zero, because there's always a gap between checking the lease and acting on it. An epoch per ownership closes it: duplicates may still *execute*, but the stale epoch's writes are **rejected**."

"One subtlety I'd stress: the epoch has to be checked **in two places** – when taking work and when reporting it done. A worker can fetch job ids, pause, lose the shard, wake up and report completion with stale ids. Checking only at fetch time lets that through. My own fuzz missed this until I made it hold job ids across a handover – then the mutation was caught 262 of 300 times."

"Third, time itself. 'Next run equals last run plus 24 hours' is correct until daylight saving. People assume it's wrong for one day. I measured a year: **220 of 365 days run at the wrong local time**, because once DST shifts you, you stay shifted until the next transition – so a 9am report goes out at 10am for six months. The fix is computing the next run from the **calendar** every time, never from the last run. And there are two edge cases with no correct answer – the hour that doesn't exist in spring and the hour that happens twice in autumn – so that choice belongs in the job definition, not the scheduler."

"Seven invariants, a thousand runs, 419,000 job runs, zero violations, five of six mutations caught. The sixth is honest: computing intervals as 'last plus interval' breaks no invariant because it still moves forward – it's a **drift** property, evidenced by the DST table rather than the fuzz."

"The invariant I'd highlight is number six, because it caught silent permanent loss. My `due()` scanned the current bucket and two behind it – a reasonable tolerance window. But if a shard has no owner for longer than that window, its jobs fall behind it and are **never returned again**. No error, no alert; those jobs simply never run. Adding a per-shard scan floor fixed it, and the effect was not subtle: job runs went from 2,407 to **418,913** across the same seeds. My system had been silently losing 99% of its work while six invariants declared it correct. **'Nothing bad happened' invariants aren't enough – you need a 'something must happen' one.**"

15. Follow-ups – dependencies, priorities, బహుళ ప్రాంతాలు

"ఒక job ఇంకొకటి పూర్వయ్యాక నడవాలంటే?"

అప్పుడు ఇది scheduler కాదు, **workflow engine** (Airflow, Temporal). తేడా §5 lo స్పష్టం: scheduler యొక్క shard key `hash(jobId)` – jobs స్వతంత్రం కాబట్టి. Dependencies ఉంటే ఒక DAG lo ఉన్న jobs **ఒక చోట** ఉండాలి (లేదా వాటి మధ్య coordination కావాలి), మరియు అది మీ ఏకరీతి పంపిణీని విరగొడుతుంది. ఆచరణ: **DAG ని ఒక యూనిట్ గా shard చేయడం** – `hash(dagId)`, `hash(jobId)` కాదు.

"కొన్ని jobs ముఖ్యమైనవి అయితే?"

ఒక్కో shard lo ఒక priority queue. కానీ ఒక హెచ్చరిక: §4 lo చూసిన అర్థరాత్రి భారం – అప్పుడు **తక్కువ ప్రాధాన్యత jobs గంటల తరబడి ఆగిపోతాయి**. LLD Deep 22 §11 మరియు LLD Deep 25 §16 lo చూసిన అదే starvation. పరిష్కారం అదే: **ఎదురుచూసిన సమయాన్ని బట్టి ప్రాధాన్యత పెంచడం**.

"Jobs ని బహుళ ప్రాంతాల్లో నడపాలంటే?"

ఒక job **ప్రపంచవ్యాప్తంగా ఒక్కసారే** నడవాలి. అంటే shard ownership ప్రాంతాల మధ్య coordinate కావాలి – మరియు ప్రాంతాల మధ్య network partition అయితే, రెండు ప్రాంతాలూ "నేనే యజమాని" అనుకుంటాయి. §8 యొక్క epoch ఇక్కడ **సరిపోదు**, ఎందుకంటే epoch counter కూడా విభజించబడుతుంది. కావాల్సింది ఒక **ఏకైక coordination service** (etcd/ZooKeeper) లేదా ప్రతి shard కి ఒక **స్థిర ప్రాంతం** – మరియు ఆ ప్రాంతం పడిపోతే ఆ shards **ఆగిపోతాయి** (fail-closed). చెల్లింపుల వంటి jobs కి అది సరైన ఎంపిక.

"10 కోట్ల jobs యొక్క shard map ఎక్కడ ఉంటుంది?"

`hash(jobId) % N` ఒక **function**, ఒక పట్టిక కాదు – కాబట్టి దాన్ని ఎక్కడా నిల్వ చేయనవసరం లేదు. కానీ `N` మారితే (shards పెంచితే) **అన్ని jobs తరలిపోతాయి**. అందుకే ఆచరణలో **consistent hashing** లేదా **చాలా**

ఎక్కువ స్థిర shards (ఉదా. 4,096) వాడి, వాటిని workers కి కేటాయించడం – shards సంఖ్య మారదు, కేటాయింపు మాత్రమే మారుతుంది.

"ఒక job చాలాసేపు నడిస్తే shard lease గడువు తీరుతుందా?"

తీరకూడదు – heartbeat job నడుస్తుండగా కూడా కొనసాగాలి. LLD Deep 25 §8 lo ఇదే: **lease "పని ఎంతసేపు" అని కొలవకూడదు, "worker బతికున్నాడా" అని కొలవాలి.** కానీ ఇక్కడ ఒక అదనపు విషయం: ఒక పొడవైన job ఆ shard యొక్క మిగతా jobs ని ఆపకూడదు – కాబట్టి job execution shard ownership నుంచి వేరుగా ఉండాలి (LLD Deep 26 §8 యొక్క bulkhead).

16. ఏమి నేర్చుకున్నాం

1. సగటు ఒక అబద్ధం – మళ్ళీ. "సెకనుకి 39 jobs" అనే సంఖ్య నిజం, మరియు అది ఎప్పుడూ జరగదు. **97.4% సెకన్లు ఖాళీ, ఒక సెకనులో 2.35 కోట్లు.**

2. Shard key ఏకరీతిగా పంచబడిన దాని మీద ఉండాలి. కాలం ప్రకారం shard చేయడం సహజంగా అనిపిస్తుంది, మరియు అది సరిగ్గా అత్యంత చెడ్డ ఎంపిక – ఎందుకంటే **కాలమే skew ఉన్న చోటు.**

3. తప్పు shard key తో shards పంచడం సహాయపడదు – అది ఖాళీ machines ని పెంచుతుంది. 16 → 1,024 shards, అసమతుల్యత 3.25 లక్షల నుంచి **2.08 కోట్ల** రెట్లు.

4. Lease ఒక కిటికీని చిన్నది చేస్తుంది, మూయదు. 1.02% → 0.07%. దాన్ని మూసేది **epoch**, మరియు అది రెండు చోట్లా ఉండాలి.

5. "చివరిది + 24 గంటలు" ఒక రోజు తప్పు కాదు – 220 రోజులు. ఎందుకంటే DST మార్పు తర్వాత ఆ జరుగుదల శాశ్వతంగా ఉండిపోతుంది.

6. కొన్ని ప్రశ్నలకి సరైన జవాబు లేదు. "లేని 2:30 కి ఏమి చెయ్యాలి" అనేది ఒక product నిర్ణయం, మరియు అది job నిర్వచనంలో ఉండాలి.

7. "ఏదీ తప్పు జరగలేదు" నియమాలు సరిపోవు. నా ఆరు నియమాలూ నెగ్గుతుండగా system **99.4% jobs ని మౌనంగా పోగొడుతూ ఉంది.** "ఏదీ జరగాలి" అనే నియమం (overdue job తప్పుక కనిపించాలి) జోడించాక – job runs **2,407 → 4,18,913.**

ఈ **DOC** నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

ఈ **problem** lo మూడు విరుపులూ ఒకే తప్పు: కాలాన్ని ఒక సాధారణ సంఖ్యగా భావించడం.

§4 lo కాలం ఏకరీతిగా పంచబడదు (23.5% ఒకే సెకనులో).

§7 lo కాలం నమ్మదగినది కాదు (lease గడువు మరియు వాస్తవం మధ్య అంతరం).

§10 lo కాలం సమానంగా సాగదు (ఒక రోజు 23 గంటలు).

Scheduler అనేది కాలాన్ని నిర్వహించే వ్యవస్థ – మరియు కాలం మీరు అనుకున్నదాని కంటే చాలా వింతగా ప్రవర్తిస్తుంది.